

Distributed Sensor Hub

Final Report for the Distributed Systems Course 2025/2026

Alex Santini

`alex.santini2@studio.unibo.it`

April 24, 2026

Distributed Sensor Hub is a fully decentralized peer-to-peer platform for smart-city sensor data collection and replication, designed to operate across multiple nodes without any central coordinator. Each node may host multiple local sensors, ingest their readings through a pluggable sensor interface to support different sensor types, and maintain a local replica of the shared system state.

The platform disseminates sensor updates through push-pull gossip and resolves conflicts with deterministic timestamp-based Last-Writer-Wins merging, providing eventual consistency across replicas. In the demo setup, sensor streams are simulated for reproducibility, but the same ingestion model can be extended to real devices through dedicated adapters.

To remain operational in realistic distributed environments, the system combines gossip-based membership, phi-accrual failure detection, and anti-entropy recovery after crashes or temporary network partitions. In CAP terms, the project prioritizes availability and partition tolerance over strong consistency, while still guaranteeing convergence once communication is restored.

Fault-injection experiments on multi-node deployments show stable recovery and consistent replica alignment after failures and partitions, supporting the suitability of the approach for decentralized smart-city monitoring scenarios.

Disclaimer

During the development of the project and the preparation of this report, the author used Large Language Models (LLMs) as auxiliary tools for limited coding support, brainstorming, and improving the clarity and structure of the written text.

All outputs produced with the assistance of LLMs were critically reviewed, validated, and, when necessary, revised by the author. All architectural decisions, system design choices, and implementation details were defined and verified by the author, who takes full responsibility for the final content of this report.

1 Concept

Product type. The developed software is a distributed experimental platform composed of a set of autonomous peer-to-peer services and an optional web application for monitoring. Concretely, it combines:

- a set of autonomous peer-to-peer nodes responsible for sensor ingestion, local state management, membership handling, failure detection, and replicated state synchronization;
- an optional read-only web/introspection layer for monitoring and runtime observability.

Use case description. The software manages a cluster of sensor nodes that ingest local readings, replicate sensor state across peers, and expose runtime information for monitoring. The project is intentionally designed as a controlled distributed-systems laboratory rather than as a production-ready smart-city platform. This makes it possible to focus on decentralization, eventual consistency, membership evolution, failure detection, anti-entropy recovery, and observability under faults and network partitions.

A typical execution proceeds as follows. First, a configurable number of nodes are started. Each node may host one or more local sensors and begins ingesting readings through a pluggable sensor interface. In the current demonstration setup, sensor streams are simulated for reproducibility, but the same interface can be extended to real devices through dedicated adapters. As readings are produced, nodes disseminate updates through push-pull gossip, exchange heartbeat evidence, propagate membership information, and synchronize their replicated state. Conflicts are resolved deterministically through a *Last-Write-Wins* policy based on $(ts_{ms}, origin)$, so replicas progressively converge after message delays, crashes, or temporary partitions.

Typical users are smart-city monitoring stakeholders with two main human interaction profiles:

- *observational interaction* (control-room operators): inspect live sensor values, node health, and cluster evolution from monitoring workstations;

- *operational interaction* (technical operators/developers): deploy nodes, tune configuration parameters, and execute recovery or fault-handling procedures from development or administration machines.

Users are typically located wherever the system is operated or observed, such as development environments, control-room settings, or laboratory test setups. Interaction is usually periodic rather than continuous: operators interact with the system when deploying experiments, changing configuration, injecting faults, or checking recovery, while observers access the dashboard whenever live status inspection is needed. In addition, external tools may consume the exposed API to inspect, collect, or further process the data maintained by the system.

Interaction channels and devices:

- HTTP API for programmatic interaction, monitoring, and integration with external tools;
- desktop/laptop web browser for optional dashboard-based monitoring.

Data handling and roles:

- the system does not store personal user data; it stores replicated sensor-state records, timestamps, origin identifiers, membership/liveness metadata, topology views, and introspection metrics/events as node-local runtime state;
- optional log files provide operational traces for debugging and validation on the machines running the nodes;
- main roles include: *node process* (autonomous peer), *cluster operator/developer*, and *observer client* (dashboard, tests, or external tooling).

Why distribution is needed. Distribution is a core requirement, not an implementation detail:

- autonomous data collection at multiple nodes rather than at a central coordinator;
- temporary divergence of local replicas before eventual convergence;
- decentralized membership management and peer liveness assessment through heartbeat observation and phi-accrual failure detection;
- resilience under crashes, restarts, and temporary network partitions through gossip dissemination and anti-entropy recovery;
- realistic smart-city deployment assumptions, where sensing points are naturally distributed at the edge.

The system does not focus on long-term historical analytics or centralized query processing. It focuses on the information needed to study decentralized sensor-state replication: current sensor values, timestamps, origin identifiers, membership and liveness metadata, topology knowledge, and runtime observability data.

2 Requirements Elicitation and Analysis

Requirements were elicited from the intended project goal described in section 1: build a decentralized laboratory platform for distributed sensor ingestion, replicated state propagation, fault observation, and monitoring. The elicitation process therefore focused on the observable services the platform must provide to operators and observers, on the quality attributes expected from a distributed-system prototype, and on the practical constraints imposed by course evaluation and reproducible experimentation.

2.1 Glossary

- **Node:** autonomous peer process participating in the cluster and executing the full runtime stack.
- **Peer:** another node known by a node through bootstrap, heartbeat exchange, or gossip dissemination.
- **Sensor reading:** atomic observation produced by a local sensor source, including sensor identifier, value, timestamp, and origin.
- **Replicated sensor state:** node-local view of the cluster-wide latest known sensor values after merge.
- **Membership view:** node-local knowledge about known peers and their liveness status.
- **Observer:** human user or external tool consuming read-only monitoring data.
- **Operator:** human user responsible for starting nodes, configuring experiments, and injecting or observing failures.
- **Convergence:** condition in which reachable replicas expose the same winning sensor values after enough communication.
- **Partition:** temporary communication disruption that prevents some nodes from exchanging messages with the rest of the cluster.

2.2 Functional Requirements

FR1 – Multi-node autonomous operation. The system shall allow multiple nodes to start independently and participate in the same distributed deployment without requiring a central coordinator.

Acceptance criterion. When a cluster of at least two configured nodes is started, each node becomes operational as an autonomous process and exposes its local service interfaces without depending on a single master node.

FR2 – Local sensor ingestion. Each node shall be able to host one or more local sensor sources and ingest their readings into the distributed system state.

Acceptance criterion. Given at least one configured sensor on a node, the node eventually exposes fresh sensor records associated with its own origin through the read API or introspection API.

FR3 – Cluster-wide sensor-state dissemination. The system shall propagate sensor updates among nodes so that readings produced on one node become visible on other reachable nodes.

Acceptance criterion. If a node generates new sensor readings while communication paths to other nodes are available, those readings eventually appear in the replicated sensor-state view exposed by the other reachable nodes.

FR4 – Deterministic conflict resolution. The system shall resolve concurrent or conflicting updates in a deterministic way so that replicas can converge to the same winning value for each sensor record.

Acceptance criterion. When two or more updates compete for the same logical sensor record, all nodes that receive the same set of updates eventually expose the same winner for that record.

FR5 – Membership discovery and peer knowledge propagation. The system shall allow nodes to discover peers and maintain a distributed view of cluster membership.

Acceptance criterion. After startup and sufficient message exchange, a node exposes a membership view containing the peers it has discovered directly or indirectly.

FR6 – Liveness observation. The system shall provide operators with a node-local assessment of peer liveness so that missing or degraded peers can be observed during experiments.

Acceptance criterion. If a known peer becomes unreachable for long enough, at least one observing node eventually reports a degraded or failed status for that peer in its membership/introspection output.

FR7 – Recovery after crashes or temporary disconnections. The system shall allow a node that restarts or reconnects after temporary unavailability to recover the replicated state needed to rejoin normal operation.

Acceptance criterion. After a node crash or temporary isolation, once communication is restored the recovering node eventually exposes a replicated state aligned with the rest of the reachable cluster.

FR8 – Read-only observability interface. The system shall provide a read-only programmatic interface for observing sensor state, membership, topology-related information, events, and operational metrics.

Acceptance criterion. A client can query documented HTTP endpoints and obtain structured snapshots of the node's current distributed-system view without modifying runtime state.

FR9 – Human-oriented monitoring support. The system shall support human observation of the running cluster through a dashboard or equivalent visual monitoring interface.

Acceptance criterion. An observer can inspect live cluster information from a web browser by connecting to the exposed read-only API of a node.

2.3 Non-Functional Requirements

NFR1 – Decentralization. Normal operation shall not depend on any central coordinator or single control component.

Acceptance criterion. The system continues to ingest local readings, maintain local state, and serve read-only observability data on surviving nodes even if one arbitrary peer becomes unavailable.

NFR2 – Eventual consistency. The replicated sensor state shall provide eventual, not immediate, consistency across nodes.

Acceptance criterion. In the absence of new conflicting updates and after communication is restored, reachable replicas converge to the same winning values for the same sensor records.

NFR3 – Partition tolerance with degraded semantics. The system shall remain available for local operation during temporary network partitions, accepting that different partitions may temporarily diverge.

Acceptance criterion. During a partition, nodes in each connected component continue local ingestion and observation; after the partition heals, their replicas eventually converge again.

NFR4 – Fault observability. The system shall make failures and recovery visible enough to support distributed-systems experimentation and debugging.

Acceptance criterion. Operators can inspect status changes, replication activity, and recovery evidence through exposed membership, event, or metric views.

NFR5 – Reproducibility of experiments. The platform shall support repeatable validation scenarios on development machines and course evaluation environments.

Acceptance criterion. Using the documented setup, an operator can start a pre-defined topology and reproduce representative convergence and recovery experiments without relying on external managed services.

NFR6 – Extensibility of sensor sources. The system shall be open to additional sensor types without changing the conceptual role of the rest of the distributed runtime.

Acceptance criterion. New sensor sources can be introduced as additional producers while preserving the same downstream ingestion, replication, and observation capabilities.

NFR7 – Operational scalability at course-project scale. The platform shall support small and medium experimental clusters rather than only a fixed two-node demo.

Acceptance criterion. The documented deployment can be instantiated with multiple nodes and still provides replicated state and observability functions for the whole experiment.

2.4 Implementation Constraints

IR1 – Self-contained deployment. The project shall be executable in a self-contained way on commodity machines used for development and evaluation, without requiring paid or institution-managed cloud infrastructure.

Justification. This constraint follows from course-project reproducibility and portability requirements.

Acceptance criterion. The system can be started locally with the provided repository artifacts and documented dependencies.

IR2 – Reproducible multi-node setup. The project shall provide a reproducible way to instantiate representative multi-node deployments for demonstrations and validation.

Justification. The report and validation activities require repeatable distributed scenarios, including fault-injection experiments.

Acceptance criterion. An operator can launch a predefined topology from the repository and repeat the same experiment configuration with consistent system roles and interfaces.

2.5 Relevant Distributed System Features

Not all classical distributed-system features have the same priority in this project. Since *Distributed Sensor Hub* is a decentralized monitoring and experimentation platform, some features are central design drivers, while others are intentionally secondary.

Transparency. Only partially relevant. The system hides low-level transport details from observers, but it does *not* aim to hide the fact that it is distributed. On the contrary, replication lag, topology evolution, liveness suspicion, and recovery dynamics are part of what the project wants to make observable. Failure transparency is therefore limited by design.

Fault tolerance and dependability. Highly relevant. Nodes may crash, restart, or become temporarily disconnected, and the platform is expected to continue operating on surviving nodes while later recovering convergence. Availability and recoverability are more important than uninterrupted global consistency. Integrity is also relevant in the sense that replicas must converge deterministically rather than drift indefinitely.

Scalability. Relevant, but at moderate scale. The project is not meant for Internet-scale deployments; however, it must scale beyond a trivial two-node example and remain meaningful on multi-node laboratory topologies. The main concern is preserving understandable behavior as the number of peers and sensor updates grows.

Security and trust. Only marginally relevant in the current scope. The platform does not manage personal data or adversarial multi-tenant access, and the main users are trusted operators and observers in controlled environments. Security hardening is therefore not a primary project driver, although it remains a possible future extension.

Resource sharing. Relevant. Nodes share distributed knowledge rather than centralized hardware resources: replicated sensor winners, membership evidence, topology

views, and operational metadata. Coordination is required to keep those shared views convergent despite asynchrony and failures.

Openness, interoperability, and heterogeneity. Relevant. The project benefits from clean interfaces between sensor producers, node runtime, and observation clients. This matters because simulated sensors are only one current source of data, while the same ingestion boundary can later support heterogeneous adapters and external tools.

Evolvability and maintainability. Highly relevant. As a course project intended to demonstrate multiple distributed-systems concerns, the platform must remain understandable, modular, and extensible, especially for adding new sensors, observability surfaces, and validation scenarios.

Performance, concurrency, and communication efficiency. Relevant, but not under hard real-time constraints. The system executes many concurrent activities such as sensor production, replication, heartbeats, gossip exchange, and HTTP observation. Efficiency matters because excessive communication can obscure convergence behavior, but the project does not target strict latency guarantees.

Economy and costs. Relevant mainly as a practical constraint. The project should run on ordinary development hardware and avoid external infrastructure dependencies, which supports reproducibility, portability, and low evaluation cost.

3 Design

This chapter explains the strategies used to meet the requirements identified in the analysis. Ideally, the design should be the same, regardless of the technological choices made during the implementation phase.

You can re-arrange the sections as you prefer, but all the sections must be present in the end

Important: try to motivate your design choices in relation to the requirements and features identified in section 2 and section 2.5.

3.1 Architecture

- Which architectural style?
 - why?

3.2 Infrastructure

- are there *infrastructural components* that need to be introduced? *how many*?
 - e.g. *clients, servers, load balancers, caches, databases, message brokers, queues, workers, proxies, firewalls, CDNs, etc.*
- how do components *distribute* over the network? *where*?

- e.g. do servers / brokers / databases / etc. sit on the same machine? on the same network? on the same datacenter? on the same continent?
- how do components *find* each other?
 - how to *name* components?
 - e.g. DNS, *service discovery*, *load balancing*, *etc.*

Component diagrams are welcome here

3.3 Modelling

- which **domain entities** are there?
 - e.g. *users*, *products*, *orders*, *etc.*
- how do *domain entities* **map to infrastructural components**?
 - e.g. state of a video game on central server, while inputs/representations on clients
 - e.g. where to store messages in an IM app? for how long?
- which **domain events** are there?
 - e.g. *user registered*, *product added to cart*, *order placed*, *etc.*
- which sorts of **messages** are exchanged?
 - e.g. *commands*, *events*, *queries*, *etc.*
- what information does the **state** of the system comprehend
 - e.g. *users' data*, *products' data*, *orders' data*, *etc.*

Class diagram are welcome here

3.4 Interaction

- how do components *communicate*? *when*? *what*?
- *which* **interaction patterns** do they enact?

Sequence diagrams are welcome here

3.5 Behaviour

- how does *each* component **behave** individually (e.g. in *response* to *events* or *messages*)?
 - some components may be *stateful*, others *stateless*
- which components are in charge of updating the **state** of the system? *when*? *how*?

State diagrams are welcome here

3.6 Data and Consistency Issues

- Is there any data that needs to be stored?
 - *what data? where? why?*
- how should *persistent data* be **stored**?
 - e.g. relations, documents, key-value, graph, etc.
 - why?
- Which components perform queries on the database?
 - *when? which queries? why?*
 - concurrent read? concurrent write? why?
- Is there any data that needs to be shared between components?
 - *why? what data?*

3.7 Fault-Tolerance

- Is there any form of data **replication** / federation / sharing?
 - *why? how does it work?*
- Is there any **heart-beating, timeout, retry mechanism**?
 - *why? among which components? how does it work?*
- Is there any form of **error handling**?
 - *what happens when a component fails? why? how?*

3.8 Availability

- Is there any **caching** mechanism?
 - *where? why?*
- Is there any form of **load balancing**?
 - *where? why?*
- In case of **network partitioning**, how does the system behave?
 - *why? how?*

3.9 Security

- Is there any form of **authentication**?
 - *where? why?*
- Is there any form of **authorization**?
 - which sort of *access control*?
 - which sorts of users / *roles*? which *access rights*?
- Are **cryptographic schemas** being used?
 - e.g. token verification,
 - e.g. data encryption, etc.

4 Implementation

Please report here all the implementation (technology-dependent) choices you made while implementing your design.

If you run out of time for this project, you may consider leaving some aspect unimplemented, and simply discuss how you would have implemented them. In this case, better would be to discuss unimplemented features in section 9.

- which **network protocols** to use?
 - e.g. UDP, TCP, HTTP, WebSockets, gRPC, XMPP, AMQP, MQTT, etc.
- how should *in-transit data* be **represented**?
 - e.g. JSON, XML, YAML, Protocol Buffers, etc.
- how should *databases* be **queried**?
 - e.g. SQL, NoSQL, etc.
- how should components be *authenticated*?
 - e.g. OAuth, JWT, etc.
- how should components be *authorized*?
 - e.g. RBAC, ABAC, etc.

4.1 Technological details

- any particular *framework* / *technology* being exploited goes here

5 Validation

5.1 Automatic Testing

- how were individual components *unit-tested*?
- how was communication, interaction, and/or integration among components tested?
- how to *end-to-end-test* the system?
 - e.g. production vs. test environment
- for each test specify:
 - rationale of individual tests
 - how were the test automated
 - how to run them
 - which requirement they are testing, if any

recall that *deployment automation* is commonly used to *test* the system in *production-like* environment

recall to test corner cases (crashes, errors, etc.)

5.2 Acceptance test

- did you perform any *manual* testing?
 - what did you test?
 - why wasn't it automatic?

6 Deployment

- should one install your software from scratch, how to do it?
 - provide instructions
 - provide expected outcomes
- what software should be installed on the machines to run your project? which versions?
- should one set environment variables or configuration files?
- if you're using containerization (e.g. Docker), describe how you engineered your deployment scrips (e.g. `docker-compose.yml` files)

7 User Guide

- how to use your software?
 - provide instructions
 - provide expected outcomes
 - provide screenshots if possible

8 Self-evaluation

- An individual section is required for each member of the group
- Each member must self-evaluate their work, listing the strengths and weaknesses of the product
- Each member must describe their role within the group as objectively as possible.

It should be noted that each student is only responsible for their own section.

9 Future works

Possible future extensions include stronger security mechanisms, persistent storage for historical sensor data, richer external integrations, and broader experimentation with larger topologies or heterogeneous sensor sources.

Another useful direction would be to integrate more advanced monitoring and analysis capabilities, including long-term trend collection, alerting, and comparative evaluation of alternative dissemination or merge strategies.